



XGBoost in Survival Analysis in Customer Churn Prediction

Mark's Quant

16. Februar 2026

Zusammenfassung

Der Erfolg von XGBoost beruht auf einer Kombination aus Algorithmen und wirkungsvollen Systemarchitekturen. XGBoost (eXtreme Gradient Boosting) gilt als einer der leistungsfähigsten Algorithmen im maschinellen Lernen. Die Anwendung von XGBoost ist besonders vorteilhaft, wenn er auf Probleme der Überlebenszeitanalyse angewendet wird. Hier treffen zwei Welten aufeinander, die klassische statistische Überlebenszeitanalyse und der moderne Methode des Machine Learning des Gradient Boosting. Durch die Verbindung der zwei Welten wird es erst ermöglicht, dass zeitliche Abhängigkeiten und Zensierung direkt in das Framework von XGBoost integriert werden können. Im Kontext der Kundenabwanderung zeigt sich die besondere Relevanz dieser Kombination, denn Standard-Klassifikationsmodelle wie Nicht-parametrische Modelle (Kaplan-Meier) oder Semiparametermodelle (Cox-Proportional-Hazards) würden die Information ignorieren, wann ein Kunde abgewandert ist oder dass viele Kunden zum Zeitpunkt der Analyse noch aktiv sind. Survival-Modelle mit XGBoost haben den Vorteil, dass genau diese zeitliche Dimension berücksichtigt und dadurch präzisere Vorhersagen liefert.

1 XGBoost Orientierung

Die Kombination aus XGBoost, dass eines der leistungsstärksten ML-Modelle für tabellarische Daten ist und der Überlebensanalyse (Survival Analysis) ist sehr mächtig. Diese Verbindung ermöglicht es nicht-lineare Zusammenhänge zu modellieren, die die klassische Modelle wie das Cox-Proportional-Hazards-Modell (Cox PH) oft ignorieren.

Die Hauptgründe für XGBoost in Survival Analysis

- Unterstützung spezifische beobachtete Daten für Survival-Analyse
- Hohe Vorhersagegenauigkeit und Robustheit
- Effiziente Handhabung großer und hochdimensionaler Datensätze

- Interpretierbarkeit und Feature-Importance (Eingabevariable des Modells und die Maßeinheit der Beeinflussung)
- Flexibilität durch `xgboost`-Bibliothek und Interpretierbarkeit
- Anwendungen jenseits der Standard-Regression wie z.B. Klassifikation mit komplexen Algorithmen.

XGBoost macht die Survival-Analyse zugänglicher und verbessert das Ergebnis, macht genauere Vorhersagen und ist skalierbarer. Das ML-Toll XGBoost findet sowohl in der quantitativen Gesundheitsforschung als auch in der Customer-Churn-Prediction Anwendung, da es durch seine mathematische Robustheit zuverlässige und präzise Ergebnisse liefert.

1.1 Was ist Churn?

Churn (auch Kundenabwanderung oder Kundenfluktuation) beschreibt den Prozess, bei dem Kunden ihre Geschäftsbeziehung zu einem Unternehmen beenden und dessen Produkte oder Dienstleistungen nicht mehr nutzen. Quantitativ kann der Churn als Prozentsatz (Churn Rate) innerhalb eines bestimmten Zeitraums gemessen werden. Die Kennzahl des Churn ist wesentlich für die Unternehmensgesundheit, insbesondere bei Abo-Modellen, da die Gewinnung von Neukunden in der Regel teurer ist als die Bindung bestehender Kunden. Unternehmen analysieren den Churn, um Gründe für die Abwanderung zu verstehen und Strategien zur Kundenbindung zu entwickeln.

1.2 Was ist XGBoost?

XGBoost fasst viele kleine, einfache Entscheidungsbäume zu einem großen Modell zusammen. So kann er sehr genaue Vorhersagen treffen. Er wird besonders häufig für Klassifikation und Regression. XGBoost beruht auf dem Prinzip des Gradient Boosting: Eine Vielzahl schwacher Modelle (meist Entscheidungsbäume) wird schrittweise trainiert, wobei jedes neue Modell die Fehler seiner Vorgänger korrigiert.

2 Einführung XGBoost

Der Erfolg von XGBoost gründet auf einer Vielzahl algorithmischer und systemischer Optimierungen, darunter Sparsity Awareness, Weighted Sketches, effiziente Cache-Nutzung, Datenkomprimierung und Sharding. Diese Arbeit mit dem Schwerpunkt Survival Analysis in Kombination mit XGBoost bietet einen prägnanten Überblick über die Methode XGBoost (eXtreme Gradient Boosting) und ist weit verbreiteter Algorithmus für maschinelles Lernen in der Datenanalyse.

2.1 Gradient Boosting: Die Kernidee

XGBoost basiert auf dem Prinzip des Ensemble-Lernens, genauer gesagt auf dem Boosting, das durch iterative Fehlerkorrektur die Modellleistung kontinuierlich verbessert. Als Beispiel für Survival-Analyse (Zeit-zu-Ereignis-Modellierung) kann ein iterative Korrektur von Residuenfehlern angenommen werden. Betrachten wir genauer das Regressionsproblem, dabei soll die Funktion $y = x^2$ geschätzt werden, basierend auf nur wenigen Datenpunkten. XGBoost baut ein Ensemble aus schwachen Lernmodellen (Entscheidungsbäume). Es werden die folgenden systematische Berechnungssequenzen beschrieben:

- Initiales Modell : $(F_0(x))$: Starte mit einem einfachen Schätzer, z.B. dem Mittelwert der Zielwerte y .
- Iterative Korrektur: Für jedes nachfolgenden Modell $m = 1, 2, \dots, M$:
 1. Berechne die Residuen: $r_{i,m} = y_i - F_{m-1}(x_i)$
 2. Fitte ein neues Modell $h_m(x)$ (flachen Baum) auf diese Residuen.
 3. Aktualisiere das Gesamtmodell: $F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$, wobei ν die Lernrate ist (0.1 um Overfitting zu vermeiden)

Um zu verstehen, warum XGBoost „extrem“ gut ist, müssen wir die Zielfunktion (Objective Function) und deren Optimierung des mittleren Taylor-Polynoms näher betrachten. Zunächst wird die Zielfunktion definiert:

- Zielfunktion: das Resultat beim überwachten Lernen ist es, eine Zielfunktion zu minimieren, die aus zwei Teilen besteht, dem Verlust (Loss) und der Regularisierung:
 1. $\mathcal{L}(\phi) = \sum_{i=1}^n l(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k)$
 2. $l(\hat{y}_i, y_i)$: Die Verlustfunktion (Mean-Squared Error), ein Maß, wie gut das Modell die Daten erklärt.
 3. $\Omega(f)$ wird Regularisierungsterm genannt, dabei kommt XGBoost ins Spiel. Der Buchstabe „X“ in XGBoost bestraft im Gegensatz dazu die Komplexität des Modells, um Overfitting zu verhindern.
- Es wird die additive Strategie angewandt, weil nicht alle Bäume gleichzeitig trainiert werden können, der Zeitpunkt t ist die Vorhersage für eine Instanz i und wird als Formel wie folgt dargestellt: $\hat{y}^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$

Gesucht wird derjenige Baum $f(t)$, dessen Hinzufügung zu den bestehenden Vorhersagen die stärkste Reduktion des Gesamtverlusts bewirkt. Wird als Formel wie folgt dargestellt:

$$\mathcal{L}^t = \sum_{i=1}^n l(y_i, \hat{y}_i^{t-1} + f_t(x_i)) + \Omega(f_t)$$

2.1.1 Taylor-Entwicklung und warum die Nutzung im XGBoost ein großartiger Kunstgriff ist

Die Taylor-Entwicklung (eine Funktion die näherungsweise durch ein Polynom beschrieben wird) unterscheidet sich von XGBoost vom „einfachem“ Gradient Boosting. Um die Verlustfunktion allgemein (nicht nur für mittlere quadratische Fehler (mean-square-error)) optimiert werden können, nutzt XGBoost eine Taylor-Entwicklung 2. Ordnung zur Annäherung der Verlustfunktion.

Der Trick dabei: Der unterschied von „normalen“ Gradient Boosting, dabei wird jedes neue Bäumchen einfach der negative Gradient (1. Ableitung der Loss-Funktion) berechnet. Diese Vorgehensweise begrenzt insbesondere die folgenden Punkte:

- Benutzt lineare Information
- Loss-Funktion nicht exakt berücksichtigt
- Optimierung weniger stabil.

Anwendung in XGBoost durch die Verwendung von Gradient und Krümmung der Funktion:

Taylor-Entwicklung 2.Ordnung:

$$L(f + \delta) \approx L(f) + g\delta + \frac{1}{2}h\delta^2$$

Dabei ist g die 1. Ableitungswert (Gradient) und h : die 2. Ableitungswert (Hesse-Matrix (Matrix der zweiten partiellen Ableitung)). Diese Vorgehensweise ist sehr Mächtig, weil beim Training XGBoost jedesmal fragt:

„Wenn der Split hier gemacht wird, wie verbessert das meine Loss-Funktion?“

Approximation zweiter Ordnung: XGBoost berechnet jeden Split analytisch mit den Schwerpunkten:

- starke Loss verbessert
- wie optimale Vorhersage aussieht. Funktioniert fast mit jeder Verlustfunktion durch Taylor Entwicklung, also auch für Log-Loss, Poisson-Loss, ...

Zweite Ableitung ist entspricht einer Verbesserung der Stabilität:

2. Ableitung kann interpretiert werden „wie stark ist die Verlustfunktion gekrümmt“

XGBoost nutzt die zweite Ableitung und demnach die Krümmungsinformation:

- um zu große Update zu vermeiden

- Lernschritte zu „dämpfen“
- Overfitting zu reduzieren
- Konvergenz zu beschleunigen

Die Taylor Approximation kann der XGBoost Algorithmus optimale Werte direkt ausrechnen, im Unterschied zu dem klassischen Gradienten Boosting, dabei werden die Werte iterativ geschätzt, als Formel ausgedrückt.

$$w^* = -\frac{\sum g_i}{\sum h_i + \lambda}$$

XGBoost nutzt eine Taylor-Entwicklung zweiter Ordnung zur Annäherung der Verlustfunktion, mit der Approximation und wird mathematisch wie folgt beschrieben:

$$\mathcal{L}^t \approx \sum_{i=1}^n \left[(y_i, \hat{y}_{t-1}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Dabei sind Terme g_i und h_i entscheidend für den Algorithmus. g_i (ist der Gradient (1. Ableitung der Verlustfunktion), die Richtung des größten Anstiegs bzw. Abfalls des Fehlers beschreibt), h_i (Term der Hesse-Matrix (2. Ableitung der Verlustfunktion), die man sich grafisch als Krümmung vorstellen kann). Wir können folgendes festhalten. XGBoost „versteht“ also durch die 2. Ableitung (h_i) die „geografische“ Landschaft der Fehlerfunktion besser als Machine Learning Verfahren und es kann aggressiver lernen, wo die Krümmung geringer ist und vorsichtiger sein, wo sie stärker ist.

2.1.2 Warum Standard-Verlustfunktionen in der Survival-Analyse versagen

Survival-Analyse (auch Time-to-Event-Analyse) beschäftigt sich mit der Modellierung von der Zeit bis zum Eintritt eines Ereignis (z.B. Tod, Rezidiv, Maschinenausfall), unter der Berücksichtigung von Zensierung. Die Standard-Verlustfunktionen wie der Mean-Square-Error (MSE) oder Cross-Entropy sind für unzensierte und vollständige Daten konzipiert, jedoch scheitern aufgrund struktureller Eigenschaften der Daten.

Die Grundlegende Datenstruktur in Survival-Analyse

In der Survival-Analyse wird jede Einheit mit den Variablen i (z.B. Patient), T_i als die Zeit und einem Indikator δ_i beschrieben:

- $T_i = \min(T_i^*, C_i)$, wobei T_i^* wahres Ereigniszeit ist und C_i die Zensierungszeit (Lost-to-Follow-up)
- $\delta_i = 1$, wenn das Ereignis eintritt ($T_i = T_i^*$) und $\delta_i = 0$, wenn zensiert ($T_i = C_i$), Ereignis tritt nach T_i ein.

Die Verteilung von T_i wird durch die Überlebensfunktion:

$$S(t) = P(T_i^* > t)$$

oder der Hazard-Funktion:

$$h(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t \leq T_i^* < t + \Delta t | T_i^* \geq t)}{\Delta t}$$

oder der Dichtefunktion

$$f(t) = -\frac{d}{dt}S(t)$$

beschrieben. Denn jede Funktion zur Modellierung von der Zufallsvariable T beschreibt eine nützliche Perspektive. Die Dichte $f(t)$ zeigt die absolute Eintrittswahrscheinlichkeit pro Zeiteinheit, die Überlebensfunktion $S(t)$ die Wahrscheinlichkeit, dass das Ereignis noch nicht eingetreten ist und die Hazard-Funktion $h(t)$ das bedingte Risiko in einem kleinen Zeitintervall gegebenen Überlebens bis t .

Definition und Ziel der Standard - Verlustfunktionen

Die Standard-Verlustfunktionen hat das Ziel die Minimierung eines Fehler zwischen beobachteten und vorhergesagten Werten abzubilden:

- MSE für Regression:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i(\theta))^2$$

- Cross-Entropy für Klassifikation:

$$L(\theta) = - \sum_i y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)$$

Diese Funktionen sind konvex und differenzierbar, das ist die Voraussetzung, dass

- alle y_i sind vollständig beobachtbar,
- die Fehler sind symmetrisch und unabhängig von der Zensierung.

Die Herleitung des Versagens

Annahme: Wir setzen voraus, dass die beobachtete Ereigniszeit durch ein gängiges Überlebensmodell (z.B. exponentiell, Weibull, Cox) beschrieben werden kann. Weiterhin nehmen wir nicht-informative Zensierung an, d.h. die Ursache und der Zeitpunkt der Zensierung liefern keine zusätzliche Information über die wahre Ereigniszeit T_i^* , außer derjenigen, die bereits durch die beobachteten Kovariaten beschrieben werden